



UNIVERSIDAD DE MONTERREY

Práctica 14

Microprocesadores

M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo García Mat: 612348

Nombre: Ricardo Garza Benítez Mat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 23 de junio del 2026

Introducción

Los servomotores son actuadores ampliamente utilizados en robótica, automatización y sistemas de control de posición, ya que permiten girar su eje a un ángulo específico con gran precisión y repetibilidad. A diferencia de un motor de corriente directa convencional que simplemente gira mientras recibe alimentación, un servomotor incorpora internamente un mecanismo de retroalimentación de posición, generalmente un potenciómetro acoplado al eje, que junto con un circuito de control integrado mantiene el eje en el ángulo indicado por la señal de mando. Esta señal de mando es precisamente una forma de modulación por ancho de pulso con periodo fijo de 20 ms, en la cual el ancho del pulso positivo codifica el ángulo destino.

En esta práctica se implementó el control de un servomotor SG90 directamente desde el microcontrolador PIC16F887, generando la señal de control por software mediante retardos precisos basados en el Timer1. La elección de software en lugar del módulo CCP1 es intencional: al no requerir hardware dedicado de PWM, la técnica puede replicarse en cualquier pin digital del PIC y sirve además para comprender a fondo el protocolo de posicionamiento del servo. El Timer1 operando con un prescaler de 1:8 y el cristal de 16 MHz proporciona una resolución de 2 μ s por tick, suficiente para cubrir el rango de pulsos del SG90 de 500 a 2500 μ s.

La práctica se dividió en dos actividades progresivas. En la primera, el microcontrolador barría automáticamente el rango completo de ángulos de 0° a 180° de forma continua y cíclica, sin intervención del usuario. En la segunda, se incorporó el convertidor ADC para leer la posición de un potenciómetro y mapear su valor de diez bits directamente al rango de anchos de pulso del servo, convirtiendo el sistema en un control analógico de posición en tiempo real.

Marco Teórico

Servomotor SG90 y protocolo de control por PWM

El servomotor SG90 es un microservo de plástico con un par de 1.8 kg·cm y un recorrido angular de 180 grados. Su interfaz de control se basa en una señal PWM con periodo fijo de 20 ms, equivalente a una frecuencia de 50 Hz. Dentro de ese periodo, el ancho del pulso positivo determina la posición angular del eje: un pulso de 500 μ s corresponde a 0 grados, un pulso de 1500 μ s corresponde a 90 grados (posición central) y un pulso de 2500 μ s corresponde a 180 grados. El circuito de control interno del servo compara continuamente el ancho del pulso recibido con la posición actual del potenciómetro interno y activa el motor integrado hasta que ambas coincidan, momento en que se detiene y mantiene la posición. La señal debe enviarse de forma continua para que el servo retenga activamente el ángulo contra cargas externas [1].

Generación de PWM por software con Timer1

El Timer1 del PIC16F887 es un contador/temporizador de 16 bits que puede operar como temporizador interno con prescalers de 1:1, 1:2, 1:4 y 1:8. Con el cristal de 16 MHz, el periodo de instrucción es de 250 ns ($F_{osc}/4$); aplicando el prescaler de 1:8 cada tick del Timer1 equivale a 2 μ s. Para generar un retardo de N μ s se precarga el registro TMR1H:TMR1L con el valor $65536 - (N/2)$ y se espera el desbordamiento indicado por

la bandera TMR1IF. Este mecanismo permite controlar tiempos con precisión de 2 μ s, resolución más que suficiente para la señal de servo cuya tolerancia típica es de $\pm 10 \mu$ s. La generación por software tiene la ventaja de poder usarse en cualquier pin de salida digital sin requerir el módulo CCP1, aunque ocupa el CPU durante cada retardo a diferencia del PWM por hardware [1].

Mapeo de rango ADC a ancho de pulso

El ADC del PIC16F887 produce resultados de diez bits con valores entre 0 y 1023, correspondientes a tensiones entre VSS y VDD respectivamente. Para convertir ese rango al rango de anchos de pulso del servo (500 μ s a 2500 μ s, con un span de 2000 μ s) se aplica una transformación lineal de la forma: ancho = $500 + (\text{adc} \times 2000) / 1023$. La división entera en aritmética de enteros requiere trabajar con un intermediario de 32 bits para evitar desbordamiento, ya que el producto $\text{adc} \times 2000$ puede alcanzar hasta 2 046 000, valor que excede los 65535 de un entero de 16 bits. El resultado del mapeo es un ancho en microsegundos que se entrega directamente a la función de generación del pulso, cerrando el lazo entre la posición física del potenciómetro y el ángulo del servo [1].

Timer1 del PIC16F887: registros y configuración

El Timer1 se configura mediante el registro T1CON. Los bits T1CKPS1:T1CKPS0 seleccionan el prescaler; con ambos en 1 se obtiene la división 1:8 utilizada en esta práctica. El bit TMR1CS selecciona la fuente de reloj: en cero se usa el oscilador interno Fosc/4. El bit TMR1ON arranca o detiene el temporizador. La bandera de desbordamiento TMR1IF en el registro PIR1 se activa cuando el contador pasa de 0xFFFF a 0x0000 y debe limpiarse por software antes de iniciar cada nuevo retardo. La carga del preload en TMR1H y TMR1L debe realizarse con el timer detenido para evitar que una escritura parcial genere un conteo erróneo, especialmente importante cuando los retardos son cortos y el timer puede desbordarse durante la escritura del segundo byte [1].

Conversión ADC en el PIC16F887

El módulo ADC del PIC16F887 opera con una referencia interna tomada de VDD y VSS cuando ADCON1 se configura con los bits VCFG1:VCFG0 en cero. El canal analógico se selecciona habilitando el pin correspondiente como analógico en el registro ANSEL y configurando su dirección como entrada en TRISA. Para esta práctica se utilizó AN0 (RA0), que lee la posición del potenciómetro. El reloj del ADC se configuró con Fosc/32 para garantizar que el tiempo de conversión sea mayor al mínimo de 1.6 μ s requerido por el módulo. El resultado justificado a la derecha se ensambla concatenando los dos bits de ADRESH con los ocho bits de ADRESL, formando el entero de diez bits que luego se mapea al ancho de pulso del servo [1].

Simulación en Proteus

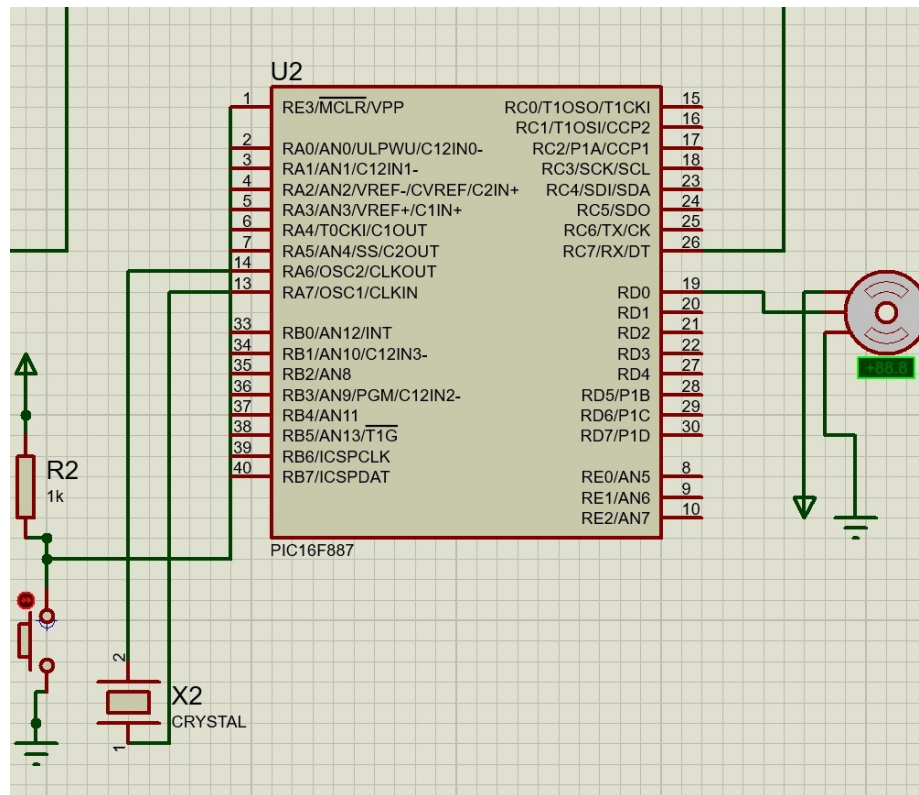
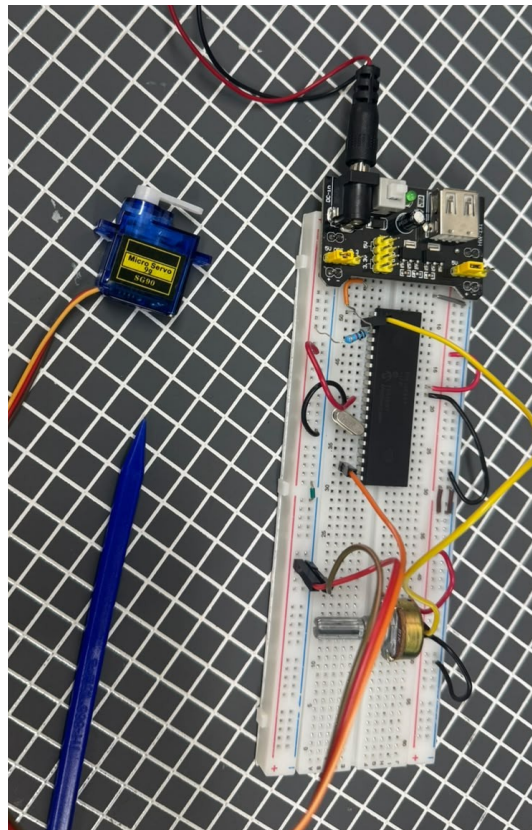


Foto del circuito en



Físico

Resultados

Actividad 1 — Barrido automático de 0° a 180°

En la primera actividad el microcontrolador generó de forma continua pulsos con anchos que variaban desde 500 μ s hasta 2500 μ s en incrementos graduales, produciendo un barrido angular completo del servo de 0° a 180°. Al llegar al extremo de 2500 μ s el ciclo se reiniciaba desde 500 μ s, repitiendo el movimiento indefinidamente. El servo SG90 respondió de manera suave y uniforme a lo largo de todo el recorrido, sin vibraciones ni saltos, lo que confirmó que la resolución de 2 μ s del Timer1 es más que suficiente para producir una trayectoria angular perceptiblemente continua.

La frecuencia de la señal de control se mantuvo estable en 50 Hz, ya que el tiempo total de cada iteración del bucle servo_pulso equivale siempre a 20 000 μ s independientemente del ancho del pulso activo: el tiempo en alto más el tiempo en bajo suman constantemente 20 ms. Este comportamiento se verificó observando el movimiento mecánico del servo, que describía un arco continuo de extremo a extremo con una velocidad angular uniforme determinada por la tasa de incremento del ancho de pulso en el código.

El circuito en la protoboard funcionó correctamente desde la primera prueba. El servo se alimentó directamente desde el regulador de la tarjeta de desarrollo, sin requerir fuente externa, y la señal de control en RD0 llegó limpia al filamento de señal del conector del SG90 sin necesidad de etapa de acondicionamiento adicional, confirmando que el nivel lógico de 5 V del PIC es compatible con el umbral de entrada del servo.

Actividad 2 — Control de posición con potenciómetro

En la segunda actividad se incorporó el potenciómetro conectado a AN0 como referencia de posición. La función ADC_Read devuelve un entero de diez bits que la función de mapeo convierte a un ancho de pulso entre 500 y 2500 μ s, el cual se entrega directamente a servo_pulso en cada iteración del bucle principal. Al rotar el potenciómetro de un extremo al otro, el servo siguió de manera inmediata y proporcional, posicionando su eje en el ángulo correspondiente a la posición mecánica del control giratorio.

Se verificó que en la posición mínima del potenciómetro el servo alcanzaba su tope de 0° y en la posición máxima llegaba a 180°, cubriendo el rango completo sin saturación visible en ninguno de los extremos. La respuesta fue suave y sin oscilaciones, lo que indica que el lazo de control interno del SG90 convergía rápidamente para cada nuevo setpoint entregado por el PIC. La actualización continua del ancho de pulso en cada trama de 20 ms garantizó que el servo siguiera fielmente cualquier movimiento del potenciómetro en tiempo real.

En conjunto, ambas actividades demostraron que es perfectamente viable generar señales de control para servomotores mediante software en el PIC16F887, sin necesidad del módulo CCP1, siempre que se disponga de un temporizador de suficiente resolución. El Timer1 con prescaler 1:8 resultó ideal para este propósito, y la aritmética de mapeo en 32 bits garantizó una distribución lineal sin errores de desbordamiento a lo largo de todo el recorrido angular.

Conclusiones Individuales

Ricardo Garza Benítez

Esta fue una de las pocas prácticas en las que el circuito funcionó exactamente como se esperaba desde la primera prueba, y eso en sí mismo fue una lección valiosa: cuando la lógica del código está bien pensada y el hardware es simple, el resultado es predecible. Lo que más me llamó la atención fue la elegancia de generar el PWM por software usando el Timer1: al precargar el registro con el complemento del número de ticks necesarios y esperar el desbordamiento, se obtiene un retardo exacto sin ningún bucle de conteo que desperdicie ciclos de CPU de forma impredecible. Ese patrón de uso del timer es algo que voy a recordar para futuras aplicaciones donde necesite temporización precisa sin comprometer el módulo CCP1.

La parte del mapeo también me resultó interesante desde el punto de vista numérico. El detalle de usar `uint32_t` para el producto intermedio antes de dividir es exactamente el tipo de error sutil que se pasa por alto fácilmente en un microcontrolador de ocho bits, donde todo parece trabajar en 16 bits pero los productos pueden exceder ese rango sin previo aviso. Que la práctica haya salido bien al primer intento fue resultado directo de haber considerado ese detalle desde el diseño del código.

Eugenio Romo García

Esta práctica fue una buena oportunidad para entender el protocolo de control de los servomotores, que a primera vista parece complejo pero en realidad se reduce a mantener una señal de 50 Hz con un ancho de pulso variable dentro de un rango bien definido. Lo que más me aportó conceptualmente fue ver cómo el servo integra su propio lazo de control: el PIC solo necesita indicar el ángulo deseado mediante el ancho del pulso y el servo se encarga de mover su eje hasta alcanzar esa posición, lo que simplifica enormemente el trabajo del microcontrolador en comparación con controlar un motor CD sin retroalimentación.

El hecho de que todo funcionara correctamente desde el primer momento reforzó la importancia de revisar cuidadosamente los parámetros de tiempo antes de programar: con los valores correctos de prescaler y preload en el Timer1, la señal generada cumplió exactamente con las especificaciones del SG90 y no fue necesario realizar ningún ajuste de último momento.

Referencias

[1] Microchip Technology. (2009). PIC16F887 Data Sheet. <https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>

[2] Tower Pro. (2010). SG90 Micro Servo Datasheet. <http://www.towerpro.com.tw/product/sg90-7/>

[3] Merino Pérez, L. A. (2012). Microcontroladores PIC: Diseño práctico de aplicaciones. McGraw-Hill.